Week-5 | Summary

Karthik Thiagarajan

1. Common 2
    1.1. Notation 2
    1.2. Labeled Dataset 2
    1.3. Data-matrix 3
    1.4. Label Vector 3
    1.5. Data-point 3
2. Supervised Learning 3
    2.1. Regression 3
    2.2. Classification 4
3. Regression 4
4. Linear Regression 5
    4.1. Model 5
    4.2. Loss Function 6
    4.3. Optimization 7
        4.3.1. Gradient 8
        4.3.2. Normal equations 8
    4.4. Gradient descent 10
    4.5. Stochastic Gradient Descent 11
    4.6. Evaluation 12
5. Geometric Perspective 12
    5.1. Best-fit surface 12
    5.2. Projections 13
6. Probabilistic Perspective 15
7. Kernel Regression 16
    7.1. Learning 16
    7.2. Prediction 18

1. Common

1.1. Notation

Scalars:

$$x_1, x_2, y_1, y_2, z_2, z_2, a, b, \alpha, \beta$$

Column vector:

$$\mathbf{x} \in \mathbb{R}^d$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix}$$

Row vector:

$$\mathbf{x} \in \mathbb{R}^d$$

$$\mathbf{x}^T = \begin{bmatrix} x_1 & \cdots & x_d \end{bmatrix}$$

Matrix:

$$\mathbf{X} \in \mathbb{R}^{d \times n}$$

1.2. Labeled Dataset

$$D = \{(\mathbf{x}_1, y_1), \ \cdots, (\mathbf{x}_n, y_n)\}$$

1.3. Data-matrix

$$\mathbf{X} \in \mathbb{R}^{d \times n}$$

- $d \rightarrow$ number of features
- $n \rightarrow$ number of data-points

$$\mathbf{X} = \begin{bmatrix} | & & | \\ \mathbf{x}_1 & \cdots & \mathbf{x}_n \\ | & & | \end{bmatrix}$$

1.4. Label Vector

$$\mathbf{y} \in \mathbb{R}^n$$

$$\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

1.5. Data-point

$$\mathbf{x}_i \in \mathbb{R}^d$$

2. Supervised Learning

2.1. Regression

Given a dataset $D = \{(\mathbf{x}_1, y_1), \cdots, (\mathbf{x}_n, y_n)\}$, where $y_i \in \mathbb{R}$, learn a function

$$f : \mathbb{R}^d \to \mathbb{R}$$

that captures the underlying relationship (pattern) between data-points and labels and generalizes it to unseen data-points. The defining characteristic of a regression problem is that the labels are continuous valued real numbers.

2.2. Classification

Given a dataset $D = \{(\mathbf{x}_1, y_1), \cdots, (\mathbf{x}_n, y_n)\}$, where $y_i \in \{1, \cdots, k\}$, learn a function

$$f : \mathbb{R}^d \to \{1, \cdots, k\}$$

that captures the underlying relationship (pattern) between data-points and labels and generalizes it to unseen data-points. The defining characteristic of a classification problem is that the labels are from a finite, discrete set. If $k = 2$, the problem is called **binary classification**. If $k > 2$, the problem is called multi-class classification.

3. Regression

If $f : \mathbb{R}^d \to \mathbb{R}$ is a potential mapping (model) from data-points to labels, we use the following metric to measure the goodness of the model.

$$L(f, D) = \frac{1}{2}\sum_{i=1}^{n} [f(\mathbf{x}_i) - y_i]^2$$

- $y_i \rightarrow$ true label for x_i
- $f(x_i) \rightarrow$predicted label
- $f(x_i) - y_i \rightarrow$error in prediction

This is called the sum of squared errors or SSE. The 0.5 is added for mathematical convenience. Dividing this by n gives the mean squared error or MSE:

$$L(f, D) = \frac{1}{2n}\sum_{i=1}^{n} [f(\mathbf{x}_i) - y_i]^2$$

The task is to find a function f that minimizes the loss function on the training dataset while generalizing well to unseen data-points. Since the class of functions is so huge, we usually choose a more tractable set.

Almost any ML algorithm can be understood in terms of four components:

- Model
- Loss function
- Optimization
- Evaluation

We will study a specific approach called linear regression.

4. Linear Regression

4.1. Model

When f is assumed to be a linear function, the resulting solution approach is called linear regression:

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_1x_1 + \cdots + w_dx_d$$

where $\mathbf{w} \in \mathbb{R}^d$ is called the weight vector. We also call f a **linear model**. For a given data-point-label pair $(\mathbf{x}_i, y_i)$, $f(\mathbf{x}_i)$ is called the predicted label and y_i is the true label. For all data-points, the **predicted label vector** can be expressed as:

$$\widehat{\mathbf{y}} = \mathbf{X}^T\mathbf{w}$$

Expanded as:

$$\widehat{\mathbf{y}} = \begin{bmatrix} \mathbf{x}_1^T\mathbf{w} \\ \vdots \\ \mathbf{x}_n^T\mathbf{w} \end{bmatrix}$$

where, $\widehat{\mathbf{y}} \in \mathbb{R}^n$.

4.2. Loss Function

The SSE loss function with a linear model expressed in three different but equivalent forms:

<u>Vector-scalar form</u>

$$L(\mathbf{w}) = \frac{1}{2}\sum_{i=1}^{n}\left[\mathbf{w}^T\mathbf{x}_i - y_i\right]^2$$

<u>Vector-vector form</u>

If $\widehat{\mathbf{y}} = \mathbf{X}^T\mathbf{w}$, we have:

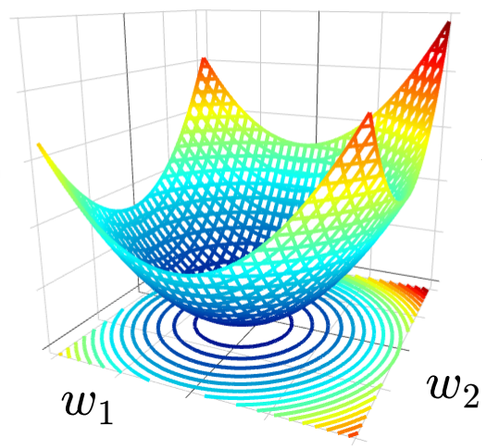
$$L(\mathbf{w}) = \frac{1}{2}||\widehat{\mathbf{y}} - \mathbf{y}||^2$$

<u>Matrix-vector form</u>

$$L(\mathbf{w}) = \frac{1}{2}||\mathbf{X}^T\mathbf{w} - \mathbf{y}||^2$$

An visualization of the squared loss and its contours for $\mathbf{w} \in \mathbb{R}^2$:

$L(w_1, w_2)$



$L(\mathbf{w}) = (w_1 - 1)^2 + (w_2 + 1)^2 + 1$

$\widehat{\mathbf{w}} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$

- The height of the surface above the $w_1 - w_2$ plane is the value of the loss function.
- The SSE loss function is convex.

4.3. Optimization

The task is to find a $\mathbf{w}$ that minimizes the loss:

$$\widehat{\mathbf{w}} = \min_{\mathbf{w}} \frac{1}{2}\sum_{i=1}^{n}\left[\mathbf{w}^T\mathbf{x}_i - y_i\right]^2$$

The SSE loss function $L(\mathbf{w})$ is convex. Hence, all its local minima are global minima.

4.3.1. Gradient

The gradient of the SSE is given as follows:

$$\nabla L(\mathbf{w}) = \begin{bmatrix} \dfrac{\partial L(\mathbf{w})}{\partial w_1} \\ \vdots \\ \dfrac{\partial L(\mathbf{w})}{\partial w_d} \end{bmatrix}$$

<u>Vector-scalar form</u>

$$\nabla L(\mathbf{w}) = \sum_{i=1}^{n} \left[\mathbf{w}^T \mathbf{x}_i - y_i\right] \mathbf{x}_i$$

<u>Matrix-vector form</u>

$$\nabla L(\mathbf{w}) = \mathbf{X}\mathbf{X}^T\mathbf{w} - \mathbf{X}\mathbf{y}$$

4.3.2. Normal equations

The system linear equations equations below are called the normal equations:

$$\mathbf{X}\mathbf{X}^T\mathbf{w} = \mathbf{X}\mathbf{y}$$

This system is consistent for any choice of $\mathbf{X}$ and $\mathbf{y}$. One special solution is given by:

$$\widehat{\mathbf{w}} = \left(\mathbf{X}^T\right)^{\dagger}\mathbf{y}$$

Why do we need the pseudo-inverse? (out of syllabus, can be skipped):

Consider the system:

$$\mathbf{A}\mathbf{x} = \mathbf{b}$$

If $\mathbf{A}\mathbf{x} = \mathbf{b}$ is not consistent, you can still hunt for an approximate solution by looking for $\mathbf{x}$ that minimizes $||\mathbf{A}\mathbf{x} - \mathbf{b}||^2$. In other words, we are trying to get as close to $\mathbf{b}$ as possible:

$$\mathbf{A}\mathbf{x} \approx \mathbf{b}$$

The resulting solution is given using the pseudo-inverse of $\mathbf{A}$:

$$\widehat{\mathbf{x}} = \mathbf{A}^{\dagger}\mathbf{b}$$

In the linear regression setting, $\mathbf{A} = \mathbf{X}^T, \mathbf{x} = \mathbf{w}, \mathbf{b} = \mathbf{y}$:

$$\mathbf{X}^T\mathbf{w} \approx \mathbf{y}$$

The solution is given by:

$$\widehat{\mathbf{w}} = \left(\mathbf{X}^T\right)^{\dagger}\mathbf{y}$$

If rank$(\mathbf{X}) = d$, then the pseudo-inverse of $\mathbf{X}^T$ can be written as:

$$\left(\mathbf{X}^T\right)^\dagger = \left(\mathbf{X}\mathbf{X}^T\right)^{-1}\mathbf{X}$$

Note that the pseudo-inverse is defined for all matrices. For more details, refer to Moore Penrose inverse. If $\mathbf{A}\mathbf{x} = \mathbf{b}$ is consistent, $\mathbf{A}^\dagger\mathbf{b}$ gives the solution that has the least norm.

$\left(\mathbf{X}^T\right)^\dagger$ is the pseudo-inverse of $\mathbf{X}^T$ and is a $d \times n$ matrix. $\widehat{\mathbf{w}}$ lies in the span of the data-points.

If rank$(\mathbf{X}) < d$, there are infinitely many solutions. Out of all of them, $\widehat{\mathbf{w}}$ is the solution that has the least norm. Any solution to the system can be written as $\widehat{\mathbf{w}} + \mathbf{w}_\perp$, where $\mathbf{w}_\perp$ is perpendicular to the span of the data-points since:

$$\mathbf{X}\mathbf{X}^T(\widehat{\mathbf{w}} + \mathbf{w}_\perp) = \mathbf{X}\mathbf{y}$$

Specifically, if rank$(\mathbf{X}) = d$ the solution is unique and is given by:

$$\widehat{\mathbf{w}} = \left(\mathbf{X}\mathbf{X}^T\right)^{-1}\mathbf{X}\mathbf{y}$$

$\widehat{\mathbf{w}}$ is sometimes termed as the least squares solution, since it is obtained by minimizing the SSE.

4.4. Gradient descent

The optimum weight vector can also be found using gradient descent. If $\mathbf{w}^{(t)}$ represents the weight vector at time t, the update is given by:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta\nabla L(\mathbf{w})\big|_{\mathbf{w}^{(t)}}$$

Substituting the gradient:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta\big(\mathbf{X}\mathbf{X}^T\mathbf{w}^{(t)} - \mathbf{X}\mathbf{y}\big)$$

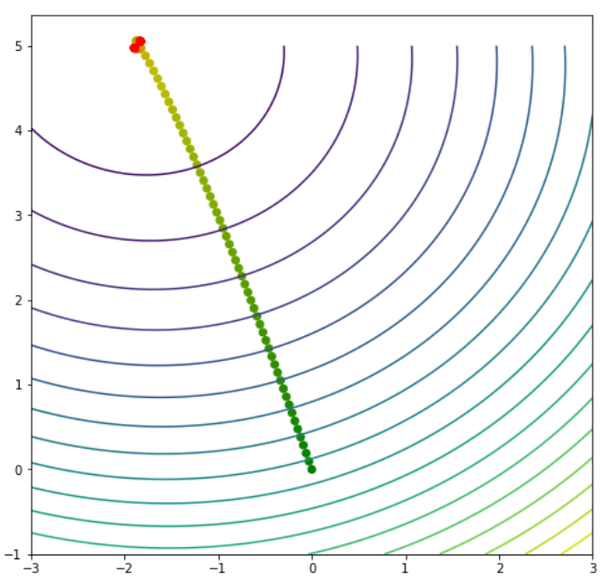
$\eta > 0$ is the learning rate. Since the objective is convex, gradient descent is guaranteed to converge for suitable choices of the learning rate η. η is called a hyperparameter (something that you choose).

4.5. Stochastic Gradient Descent

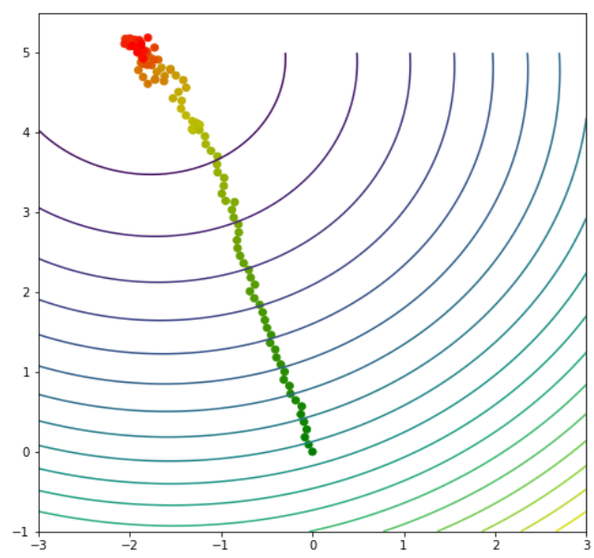
For large datasets, computing the entire gradient in one go might be costly. In such cases, a random subset (random sample) of the dataset is extracted. The approximate gradient from this subset is used in the update equation. If $\mathbf{X}' \in \mathbb{R}^{d\times m}$ represents a sample of m points:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta\Big[\big(\mathbf{X}'\big)\big(\mathbf{X}'\big)^T\mathbf{w}^{(t)} - \mathbf{X}'\mathbf{y}\Big]$$

The progress of SGD is less smooth given that we are only using an approximate version the gradient. Eventually both reach the minimum.



Gradient Descent



Stochastic Gradient Descent

4.6. Evaluation

The model's performance is evaluated on a **test dataset** which is a collection of data-points not seen during training. A good performance metric is the root mean squared error (RMSE):

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(\mathbf{w}^T\mathbf{x}_i - y_i\right)^2}$$

This has the same units as the label which the error more interpretable.

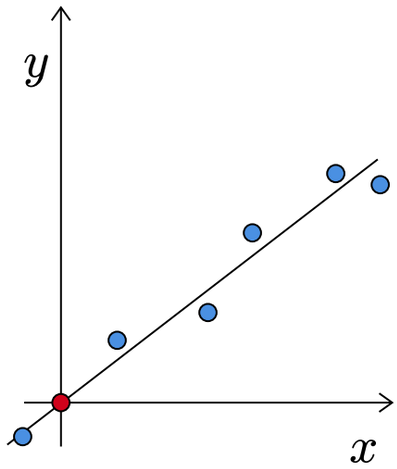
5. Geometric Perspective

5.1. Best-fit surface

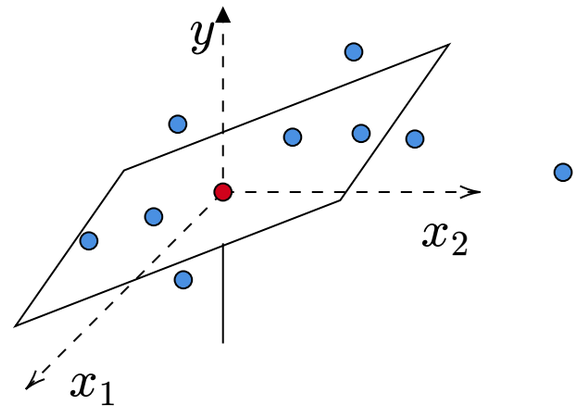
The geometry of the relationship between predicted label and features in linear regression takes the following form in different dimensions:

d	**Space**	**Object**	**Interpretation**
1	$\mathbb{R}^2$	Line passing through the origin	Best-fit line
2	$\mathbb{R}^3$	Plane passing through the origin	Best-fit plane
$\geqslant 3$	$\mathbb{R}^{d+1}$	Hyperplane passing through the origin	Best-fit hyperplane

Here is a visualization of the first two cases:



$\widehat{y} = wx$



$\widehat{y} = w_1x_1 + w_2x_2$

Each blue point here represents $(\mathbf{x}_i, y_i)$, where y_i is the true label. Note that the point $(\mathbf{x}_i, y_i)$ need not lie on the hyperplane. However, the point $(\mathbf{x}_i, \widehat{y}_i)$ will lie on the hyperplane.

5.2. Projections

<u>Feature vectors</u>

If all the feature values for a given feature are added as components of a vector, we get a feature vector. For example, $\mathbf{r}_j$ will have the values for the jth feature for all data-points:

$$\mathbf{r}_j \in \mathbb{R}^n$$

We can now look at $\mathbf{X}$ in two ways:

<u>Columns made up of data-points</u>

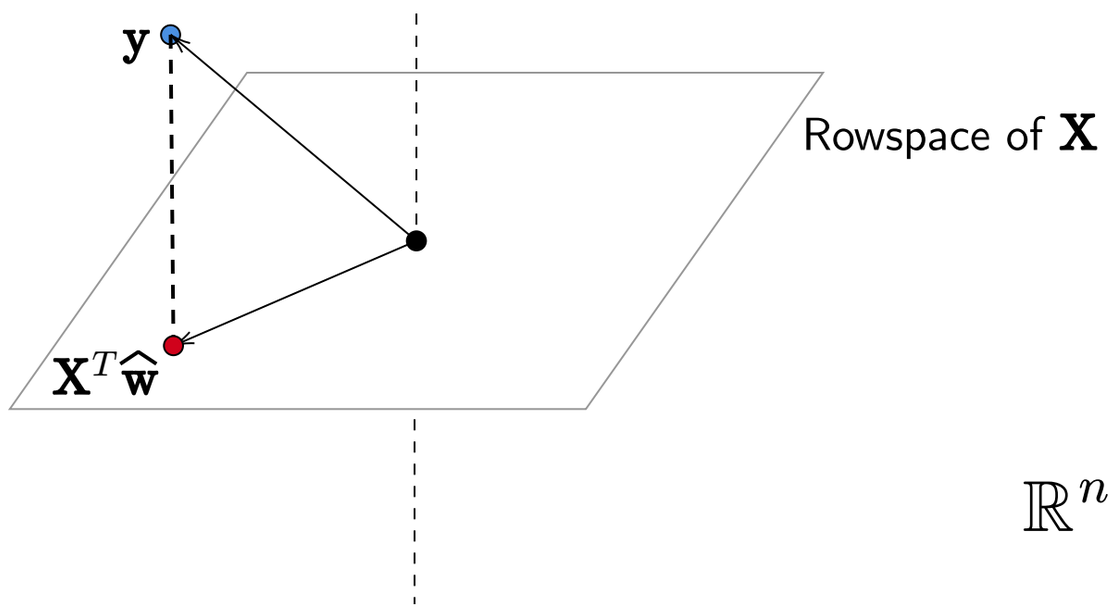
$$\mathbf{X} = \begin{bmatrix} | & & | \\ \mathbf{x}_1 & \cdots & \mathbf{x}_n \\ | & & | \end{bmatrix}, \mathbf{X}^T = \begin{bmatrix} - & \mathbf{x}_1^T & - \\ & & \\ - & \mathbf{x}_n & - \end{bmatrix}$$

<u>Rows made up of feature vectors</u>

$$\mathbf{X} = \begin{bmatrix} - & \mathbf{r}_1^T & - \\ & \vdots & \\ - & \mathbf{r}_d^T & - \end{bmatrix}, \mathbf{X}^T = \begin{bmatrix} | & & | \\ \mathbf{r}_1 & \cdots & \mathbf{r}_d \\ | & & | \end{bmatrix}$$

The row space of $\mathbf{X}$, which is the same as the column space of $\mathbf{X}^T$, is the span of the feature vectors. The SSE objective can be understood as follows. Find a point in the row space of $\mathbf{X}$ that is closest to $\mathbf{y}$:

$$\min_{\mathbf{w}} \quad ||\mathbf{X}^T\mathbf{w} - \mathbf{y}||^2$$



The required point is the projection of $\mathbf{y}$ on the row space of $\mathbf{X}$. This means that $\mathbf{y} - \mathbf{X}^T\mathbf{w}$ is orthogonal to the row space of X:

$$\mathbf{X}\left(\mathbf{y} - \mathbf{X}^T\widehat{\mathbf{w}}\right) = \mathbf{0}$$

This leads us back to the normal equations, $\mathbf{X}\mathbf{X}^T\widehat{\mathbf{w}} = \mathbf{X}\mathbf{y}$. The projection of $\mathbf{y}$ onto the row space of $\mathbf{X}$ is given by $\left(\mathbf{X}^T\right)^\dagger\mathbf{y}$.

6. Probabilistic Perspective

We assume that y_i is generated by adding a noise term to $\mathbf{w}^T\mathbf{x}_i$:

$$y_i = \mathbf{w}^T\mathbf{x}_i + \epsilon_i$$

Assuming $\epsilon_i \sim \mathcal{N}\left(0, \sigma^2\right)$, we have:

$$y_i \mid \mathbf{x}_i \sim \mathcal{N}\left(\mathbf{w}^T\mathbf{x}_i, \sigma^2\right)$$

We can now use MLE to estimate $\mathbf{w}$.

<u>Likelihood</u>

$$L(\mathbf{w}) \propto \prod_{i=1}^{n} \exp\left[\frac{-\left(y_i - \mathbf{w}^T\mathbf{x}_i\right)^2}{2\sigma^2}\right]$$

<u>Log-likelihood</u>

After collecting terms that do not affect the optimization in c:

$$l(\mathbf{w}) = -\frac{1}{2\sigma^2}\sum_{i=1}^{n}\left(y_i - \mathbf{w}^T\mathbf{x}_i\right)^2 + c$$

Maximizing the log-likelihood is the same as minimizing the negative log-likelihood. Therefore we end up with the same optimization problem that we saw before:

$$\min_{\mathbf{w}} \sum_{i=1}^{n} \left(y_i - \mathbf{w}^T \mathbf{x}_i\right)^2$$

Minimizing the SSE is equivalent to maximizing the likelihood under a zero mean, Gaussian noise assumption.

7. Kernel Regression

7.1. Learning

Consider the non-linear transformation of the features:

$$\phi : \mathbb{R}^d \to \mathbb{R}^D$$

The transformed dataset is:

$$\{(\phi(\mathbf{x}_1), y_1), \ \cdots, (\phi(\mathbf{x}_n), y_n)\}$$

The transformed data-matrix is:

$$\phi(\mathbf{X}) \in \mathbb{R}^{D \times n}$$

We can now perform regression in the transformed feature space. The model is:

$$f : \mathbb{R}^D \to \mathbb{R}$$

$$f(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x})$$

The loss function becomes:

$$L(\mathbf{w}) = ||\phi(\mathbf{X})^T\mathbf{w} - \mathbf{y}||^2$$

Since $\mathbf{w}$ lies in the span of the data-points, for coefficient vector $\boldsymbol{\alpha} \in \mathbb{R}^n$

$$\mathbf{w} = \phi(\mathbf{X})\boldsymbol{\alpha} = \begin{bmatrix} | & & | \\ \phi(\mathbf{x}_1) & \cdots & \phi(\mathbf{x}_n) \\ | & & | \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_n \end{bmatrix}$$

The loss now becomes:

$$L(\mathbf{w}) = ||\phi(\mathbf{X})^T\phi(\mathbf{X})\boldsymbol{\alpha} - \mathbf{y}||^2$$

If the kernel associated with ϕ is:

$$k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$$

the kernel matrix $\mathbf{K} \in \mathbb{R}^{n \times n}$ is:

$$\mathbf{K} = \phi(\mathbf{X})^T\phi(\mathbf{X})$$

which is computed element-wise as

$$K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$$

With this, the loss function's final form is:

$$L(\mathbf{w}) = ||\mathbf{K}\boldsymbol{\alpha} - \mathbf{y}||^2$$

Minimizing the L would result in:

$$\widehat{\boldsymbol{\alpha}} = \mathbf{K}^\dagger\mathbf{y}$$

If $\mathbf{K}$ is invertible, we have

$$\widehat{\boldsymbol{\alpha}} = \mathbf{K}^{-1}\mathbf{y}$$

With this, $\widehat{\mathbf{w}} = \phi(\mathbf{X})\widehat{\boldsymbol{\alpha}}$, which cannot be explicitly known since ϕ is not explicitly computed.

7.2. Prediction

For a test data-point $\mathbf{x}_{\text{test}}$, the predicted label is:

$$\widehat{y} = \phi(\mathbf{x}_{\text{test}})^T\widehat{\mathbf{w}}$$

Using $\widehat{\mathbf{w}} = \phi(\mathbf{X})\widehat{\boldsymbol{\alpha}}$, we have:

$$\widehat{y} = \phi(\mathbf{x}_{\text{test}})^T\phi(\mathbf{X})\widehat{\boldsymbol{\alpha}}$$

The product before $\widehat{\boldsymbol{\alpha}}$ is a $1 \times n$ vector:

$$\phi(\mathbf{x}_{\text{test}})^T\begin{bmatrix} | & & | \\ \phi(\mathbf{x}_1) & \cdots & \phi(\mathbf{x}_n) \\ | & & | \end{bmatrix} = \begin{bmatrix} \phi(\mathbf{x}_{\text{test}})^T\phi(\mathbf{x}_1) & \cdots & \phi(\mathbf{x}_{\text{test}})^T\phi(\mathbf{x}_n) \end{bmatrix}$$

Expanding the RHS of the predicted label:

$$\widehat{y} = \sum_{i=1}^{n} \widehat{\alpha}_i \cdot k(\mathbf{x}_{\text{test}}, \mathbf{x}_i)$$

- $\widehat{\alpha}_i$: importance of ith data-point in predicting the label for $\mathbf{x}_{\text{test}}$
- $k(\mathbf{x}_{\text{test}}, \mathbf{x}_i)$: similarity between $\mathbf{x}_{\text{test}}$ and $\mathbf{x}_i$